

Selenium IDE & DevTools Automation Guide

Step-by-step practical implementation and test scripts

Task 1 Record and Handle a JavaScript Alert

HTML Setup (Local Test Page)

Save the following code as `alert_test.html` and open it in Chrome:

```
<!DOCTYPE html>
<html>
<body>
  <h2>JavaScript Alert Demo</h2>
  <button id="alertBtn" onclick="alert('Welcome to Testing!')">Show Alert</button>
</body>
</html>
```

Selenium IDE Steps:

- 1. Open **Selenium IDE** and click **Record a new test in a new project**.
- 2. Enter the local file path or URL (e.g., `file:///C:/path/to/alert_test.html`) and click **Start Recording**.
- 3. Click the **Show Alert** button on the webpage.
- 4. Click **OK** on the alert pop-up.
- 5. Stop recording. Selenium IDE automatically captures alert handlers using the confirmation command.

Command	Target	Value
open		/path/to/alert_test.html
click	id=alertBtn	
webdriver choose ok on visible confirmation		

Task 2 Identify an iFrame using Chrome DevTools

1. Open any page containing an iFrame, or save and open this code as `iframe_demo.html` :

```
<!DOCTYPE html>
<html>
<body>
  <h2>iFrame Demo</h2>
  <iframe id="youtubeIframe" name="mediaFrame" width="560" height="315"
    src="https://www.youtube.com/embed/dQw4w9WgXcQ"></iframe>
</body>
</html>
```

2. **Right-click** directly over the embedded content (video/map) and select **Inspect**.

3. In the **Elements panel**, look up the DOM tree slightly until you find the `<iframe>` tag.

4. Note down the attributes:

- **ID:** `youtubeIframe`
- **Name:** `mediaFrame`
- **XPath / CSS Locator:** `css=iframe#youtubeIframe` or `xpath=//iframe[@id='youtubeIframe']`

Task 3 Switch to an iFrame and Verify Elements Inside

Selenium IDE requires explicit context switching to interact with elements inside an iFrame.

Command	Target	Value
open	/path/to/iframe_demo.html	
select frame	index=0 (or id=youtubeIframe)	
assert element present	css=.ytp-large-play-button	
select frame	relative=parent	

Note: Always remember to add `select frame | relative=parent` at the end to bring the execution context back to the main document.

Task 4 Handle Pop-up Windows and Verify Page Title

HTML Setup (Local Test Page)

Save as `popup_test.html` :

```
<!DOCTYPE html>
<html>
<body>
  <h2>Pop-up Window Demo</h2>
  <button id="openWinBtn" onclick="window.open('https://example.com', 'popupWindow',
'width=600,height=400')">Open Window</button>
</body>
</html>
```

Selenium IDE Script:

Command	Target	Value
open	/path/to/popup_test.html	
click	id=openWinBtn	
select window	title=Example Domain (or win_ser_1)	
assert title	Example Domain	
close		
select window	handle=\${root}	

Task 5 Navigate Multi-Frame Sites and Verify Interactions

HTML Setup (Frameset Example)

Save as `frameset_test.html` :

```
<!DOCTYPE html>
<html>
<frameset cols="30%,70%">
  <frame name="navFrame" srcdoc="<a id='link1' href='data:text/html,<h1>Page 1</h1>'
target='contentFrame'>Load Page 1</a>">
  <frame name="contentFrame" srcdoc="<h1>Welcome</h1>">
</frameset>
</html>
```

Selenium IDE Script:

Command	Target	Value
open	/path/to/frameset_test.html	
select frame	name=navFrame	
click	id=link1	
select frame	relative=parent	
select frame	name=contentFrame	
assert text	css=h1	Page 1
select frame	relative=parent	